@cpg/kubera Unified Client SDK

Typed REST client architecture, transport, retry policy, namespaces and React integration

1. SDK goals and public API

@cpg/kubera is the unified TypeScript client for CPG Billing & Distribution. It should provide stable, strongly typed namespaces, shared transport and error normalization, dynamic token injection, safe retry behavior, and direct mapping to documented API DTOs.

Security boundary: a browser SDK is not a secret store. Do not embed privileged tenant API keys in a public React bundle. Browser applications should use user/session tokens issued by the existing CPG identity system and server-enforced scopes. High-privilege tenant API credentials belong in trusted server environments.

1.1 Package exports

```
import { createKubera } from "@cpg/kubera";

const kubera = createKubera({
  baseUrl: "https://api.cpg.example.com/api/v1",
  getAccessToken: async () => auth.getAccessToken(),
  timeoutMs: 12_000,
  retry: { maxAttempts: 2, baseDelayMs: 250, maxDelayMs: 2_000 },
  onRequest: ({ requestId, method, path }) => telemetry.requestStart({ requestId, method, path }),
  onResponse: ({ requestId, status, durationMs }) => telemetry.requestEnd({ requestId, status, durationMs })
});

const products = await kubera.catalog.products.list({ status: "active", limit: 20 });
const session = await kubera.checkout.createSession({
  customerId: "cus_123",
  items: [{ priceId: "price_123", quantity: 1 }],
  successUrl: "https://app.example.com/success",
  cancelUrl: "https://app.example.com/cancel"
}, { idempotencyKey: crypto.randomUUID() });
```

1.2 Namespace map

Namespace	Methods
kubera.catalog.products	list, get, create, update, archive
kubera.catalog.prices	list, create, archive
kubera.checkout	createSession, getSession, cancelSession
kubera.orders	list, get
kubera.payments	get, listAttempts, createRefund, listRefunds
kubera.billing.subscriptions	create, get, list, cancel, resume
kubera.billing.invoices	list, get
kubera.usage	record, getEvent, getSummary
kubera.credits	getBalance, consume, listEntries, createAdjustment
kubera.entitlements	check, list, revoke, suspend
kubera.distribution.storefront	getBySlug
kubera.distribution.deliveries	get, retry
kubera.distribution.licenses	activate, list, deactivate
kubera.affiliates	create, list, getReport, listCommissions
kubera.operations.reconciliation	listRuns, startRun, getRun, listExceptions, resolveException

2. Core client implementation

2.1 Directory structure

```
packages/kubera/
  src/
    index.ts
    create-kubera.ts
    client.ts
    transport.ts
    config.ts
    errors/
      kubera-error.ts
      normalize-error.ts
    resources/
      catalog.ts
      checkout.ts
      orders.ts
      payments.ts
      billing.ts
      usage.ts
      credits.ts
      entitlements.ts
      distribution.ts
      affiliates.ts
      operations.ts
    types/
      common.ts
      catalog.ts
      checkout.ts
      payments.ts
      billing.ts
      distribution.ts
  tests/
    transport.test.ts
    retries.test.ts
    errors.test.ts
    resources.test.ts
```

2.2 Shared DTOs

```
export type RequestOptions = {
  signal?: AbortSignal;
  idempotencyKey?: string;
  timeoutMs?: number;
  headers?: Record<string, string>;
};

export type Page<T> = {
  data: T[];
  page: { nextCursor: string | null; hasMore: boolean };
};

export type ProductDTO = {
  id: string; name: string; slug: string; productType: "digital_service"|"download"|"license";
  status: "draft"|"active"|"archived"; createdAt: string; updatedAt: string;
};
export type PriceDTO = {
  id: string; productId: string; model: "one_time"|"subscription"|"usage"|"credit_pack";
  currency: string; unitAmountMinor?: number; interval?: "day"|"week"|"month"|"year";
  status: "active"|"archived";
};
export type CheckoutSessionDTO = {
  id: string; status: "pending"|"completed"|"expired"|"cancelled";
  currency: string; totalMinor: number; checkoutUrl: string; expiresAt: string;
};
export type PaymentDTO = {
  id: string; orderId: string; provider: string; amountMinor: number; currency: string;
  status: string; failureCode?: string; createdAt: string; updatedAt: string;
};
export type EntitlementCheckDTO = { allowed: boolean; reason: string; expiresAt: string | null };
```

2.3 Client configuration

```
export type RetryConfig = {
  maxAttempts: number; // Includes first attempt; recommended 1-3.
  baseDelayMs: number;
  maxDelayMs: number;
};
export type KuberaConfig = {
  baseUrl: string;
  getAccessToken?: () => Promise<string | null>;
  timeoutMs?: number;
  retry?: Partial<RetryConfig>;
  fetch?: typeof fetch;
  onRequest?: (event: { requestId: string; method: string; path: string }) => void;
```

```
  onResponse?: (event: { requestId: string; status: number; durationMs: number }) => void;
};
```

2.4 Transport and retry policy

The transport should inject the current access token for each request (not capture a token once at construction), attach a request ID, apply timeout/AbortSignal, parse the standard error envelope, and return typed JSON DTOs. Retries are conservative: retry network errors and 408/429/502/503/504 only for safe reads by default. Mutating requests retry only when the operation is protected by a server-supported idempotency key. Respect Retry-After where present.

```
export class KuberaError extends Error {
  readonly name = "KuberaError";
  constructor(
    message: string,
    readonly code: string,
    readonly status: number,
    readonly requestId?: string,
    readonly details?: Array<{ path: string; code: string; message: string }>,
    readonly retryable = false
  ) { super(message); }
}
export class KuberaNetworkError extends KuberaError {
  readonly name = "KuberaNetworkError";
}
export class KuberaTimeoutError extends KuberaError {
  readonly name = "KuberaTimeoutError";
}
export class KuberaAuthError extends KuberaError {
  readonly name = "KuberaAuthError";
}
export class KuberaValidationError extends KuberaError {
  readonly name = "KuberaValidationError";
}
export class KuberaConflictError extends KuberaError {
  readonly name = "KuberaConflictError";
}
export class KuberaRateLimitError extends KuberaError {
  readonly name = "KuberaRateLimitError";
}
export class KuberaServerError extends KuberaError {
  readonly name = "KuberaServerError";
}
```

2.5 Transport reference implementation

```
type ApiErrorEnvelope = { error?: {
  code?: string; message?: string; requestId?: string;
  details?: Array<{ path:string; code:string; message:string }>;
} };

export class HttpTransport {
  private readonly timeoutMs: number;
  private readonly maxAttempts: number;
  private readonly baseDelayMs: number;
  private readonly maxDelayMs: number;
  private readonly fetchImpl: typeof fetch;

  constructor(private readonly config: KuberaConfig) {
    this.timeoutMs = config.timeoutMs ?? 12_000;
    this.maxAttempts = config.retry?.maxAttempts ?? 2;
    this.baseDelayMs = config.retry?.baseDelayMs ?? 250;
    this.maxDelayMs = config.retry?.maxDelayMs ?? 2_000;
    this.fetchImpl = config.fetch ?? fetch;
  }

  async request<T>(
    method: string, path: string, body?: unknown, options: RequestOptions = {}
  ): Promise<T> {
    const requestId = crypto.randomUUID();
    const attempts = this.isRetrySafe(method, options) ? this.maxAttempts : 1;
    let lastError: unknown;

    for (let attempt = 1; attempt <= attempts; attempt++) {
      const controller = new AbortController();
      const timeout = setTimeout(() => controller.abort(), options.timeoutMs ?? this.timeoutMs);
      const externalAbort = () => controller.abort();
      options.signal?.addEventListener("abort", externalAbort, { once: true });
      const started = Date.now();

      try {
        const token = await this.config.getAccessToken?.();
        const headers: Record<string,string> = {
          "Accept": "application/json",
```

```
          "X-Request-Id": requestId,
          ...options.headers
        };
        if (body !== undefined) headers["Content-Type"] = "application/json";
        if (token) headers["Authorization"] = `Bearer ${token}`;
        if (options.idempotencyKey) headers["Idempotency-Key"] = options.idempotencyKey;

        this.config.onRequest?.({ requestId, method, path });
        const response = await this.fetchImpl(new URL(path.replace(/^\//, ""), this.config.baseUrl), {
          method, headers, body: body === undefined ? undefined : JSON.stringify(body),
          signal: controller.signal
        });
        this.config.onResponse?.({ requestId, status: response.status, durationMs: Date.now() - started });

        if (!response.ok) {
          const payload = await response.json().catch(() => ({})) as ApiErrorEnvelope;
          const err = normalizeKuberaError(response.status, payload, requestId);
          if (!err.retryable || attempt === attempts) throw err;
          lastError = err;
        } else if (response.status === 204) {
          return undefined as T;
        } else {
          return await response.json() as T;
        }
      } catch (error) {
        if (error instanceof KuberaError) {
          if (!error.retryable || attempt === attempts) throw error;
          lastError = error;
        } else if (controller.signal.aborted) {
          throw new KuberaTimeoutError("Request timed out or was aborted.", "TIMEOUT", 0, requestId, undefined, false);
        } else {
          lastError = new KuberaNetworkError("Network request failed.", "NETWORK_ERROR", 0, requestId, undefined, true);
          if (attempt === attempts) throw lastError;
        }
      } finally {
        clearTimeout(timeout);
        options.signal?.removeEventListener("abort", externalAbort);
      }

      const delay = Math.min(this.maxDelayMs, this.baseDelayMs * 2 ** (attempt - 1));
      // Production implementation should add jitter and honor Retry-After.
      await new Promise(resolve => setTimeout(resolve, delay));
    }
    throw lastError;
  }

  private isRetrySafe(method: string, options: RequestOptions): boolean {
    const safeRead = ["GET", "HEAD"].includes(method.toUpperCase());
    return safeRead || Boolean(options.idempotencyKey);
  }
}

function normalizeKuberaError(status: number, payload: ApiErrorEnvelope, requestId: string): KuberaError {
  const e = payload.error ?? {};
  const code = e.code ?? "HTTP_ERROR";
  const message = e.message ?? `Request failed with HTTP ${status}`;
  const retryable = [408, 429, 502, 503, 504].includes(status);
  const args = [message, code, status, e.requestId ?? requestId, e.details, retryable] as const;
  if (status === 401 || status === 403) return new KuberaAuthError(...args);
  if (status === 400 || status === 422) return new KuberaValidationError(...args);
  if (status === 409) return new KuberaConflictError(...args);
  if (status === 429) return new KuberaRateLimitError(...args);
  if (status >= 500) return new KuberaServerError(...args);
  return new KuberaError(...args);
}
```

Implementation note: the transport is a reference sketch, not a drop-in certified production client. Add URL joining tests, Retry-After parsing, jitter, browser/runtime compatibility checks, response schema validation where required, and careful timeout-versus-user-abort distinction. Do not retry a mutating operation merely because a client-generated key exists unless the server guarantees idempotency for that route.

3. Resource clients

3.1 Resource class example

```
export class CatalogClient {
  constructor(private readonly http: HttpTransport) {}

  products = {
    list: (query: { status?: string; limit?: number; after?: string } = {}) =>
      this.http.request<Page<ProductDTO>>("GET", `products${toQuery(query)}`),

    get: (id: string) =>
      this.http.request<ProductDTO>("GET", `products/${encodeURIComponent(id)}`),
```

```
    create: (dto: CreateProductDTO) =>
      this.http.request<ProductDTO>("POST", "products", dto),

    update: (id: string, dto: UpdateProductDTO, options: RequestOptions = {}) =>
      this.http.request<ProductDTO>("PATCH", `products/${encodeURIComponent(id)}`, dto, options)
  };

  prices = {
    create: (dto: CreatePriceDTO, options: RequestOptions = {}) =>
      this.http.request<PriceDTO>("POST", "prices", dto, options)
  };
}
function toQuery(values: Record<string, string | number | boolean | undefined>): string {
  const params = new URLSearchParams();
  for (const [key, value] of Object.entries(values)) if (value !== undefined) params.set(key, String(value));
  const query = params.toString();
  return query ? `?${query}` : "";
}
```

3.2 Factory and public exports

```
export function createKubera(config: KuberaConfig) {
  const http = new HttpTransport(config);
  return {
    catalog: new CatalogClient(http),
    checkout: new CheckoutClient(http),
    orders: new OrdersClient(http),
    payments: new PaymentsClient(http),
    billing: new BillingClient(http),
    usage: new UsageClient(http),
    credits: new CreditsClient(http),
    entitlements: new EntitlementsClient(http),
    distribution: new DistributionClient(http),
    affiliates: new AffiliatesClient(http),
    operations: new OperationsClient(http)
  } as const;
}
export type KuberaClient = ReturnType<typeof createKubera>;
```

4. React integration

4.1 State strategy

- Use the SDK as the transport/domain client, not as a global cache. Integrate TanStack Query or the existing CPG server-state library for caching, deduplication and invalidation.
- Server state: products, prices, orders, subscriptions, invoices, entitlements, deliveries, affiliate reports.
- Local UI state: open dialogs, selected tabs, unsaved forms, sort menus and ephemeral checkout UI.
- URL state: filters, pagination cursor, date ranges and selected entity IDs for shareable admin screens.
- Invalidate relevant query keys after mutations and webhook-driven events. Do not treat SDK response objects as permanently fresh.
- Cancellation: pass AbortSignal from query library to SDK methods. Ensure cancellation does not trigger retries.

4.2 Example

```
const kubera = createKubera({
  baseUrl: import.meta.env.VITE_CPG_API_URL,
  getAccessToken: () => auth.getAccessToken()
});

function useProducts(status = "active") {
  return useQuery({
    queryKey: ["products", status],
    queryFn: ({ signal }) => kubera.catalog.products.list({ status, limit: 50 }, { signal })
  });
}

function useCreateCheckout() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (dto: CreateCheckoutSessionDTO) =>
      kubera.checkout.createSession(dto, { idempotencyKey: crypto.randomUUID() }),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ["orders"] })
  });
}
```

5. Error handling and UI mapping

SDK error	Typical UI behavior
KuberaAuthError (401)	Refresh login/session once through identity provider; then show sign-in.
KuberaAuthError (403)	Show permission message; do not endlessly retry.
KuberaValidationError (400/422)	Map details[].path to field errors; retain form input.
KuberaConflictError (409)	Refresh current resource; explain stale edit/idempotency conflict.
KuberaRateLimitError (429)	Honor Retry-After; show a calm wait state; prevent repeated clicks.
KuberaTimeoutError	Show unknown/pending state for financial mutations; query status before allowing a second action.
KuberaNetworkError	Offer retry for safe operations; preserve form data.
KuberaServerError (5xx)	Show requestId for support; retry only when policy says safe.

6. SDK quality and release requirements

- Compile with strict TypeScript: strict, noUncheckedIndexedAccess, exactOptionalPropertyTypes where compatible.
- Publish ESM and CJS only if required by supported consumers; define package exports and semver policy.
- Generate DTO types from the OpenAPI contract or shared schema package; avoid hand-maintained duplicated definitions.
- Unit-test auth token refresh, timeout, abort, query serialization, error normalization and retry decisions.
- Contract-test every namespace against a mock OpenAPI server and integration-test at least one real sandbox flow.
- Never log access tokens, license keys, payment secrets or full sensitive payloads.
- Document browser versus server usage and credential boundaries. Use peer dependencies intentionally and avoid shipping unnecessary UI framework dependencies.
- Release with changesets/changelog, API compatibility checks and a migration policy.

7. Required tests

Area	Must-pass cases
Tenant isolation	Cross-tenant reads/writes rejected for every collection and endpoint.
Idempotency	Same key/same payload returns stable result; same key/different payload is 409.
Payments	Duplicate/out-of-order webhooks; timeout after provider success; partial refunds; reconciliation.
Billing	Duplicate renewal job; late usage; finalized invoice correction; cancellation timing.
Credits	Concurrent consumption; expired credit bucket; duplicate grant; insufficient balance.
Distribution	Duplicate fulfillment event; object storage failure; signed URL expiry; entitlement revoked before download.
Licenses	Concurrent activation at final slot; invalid key rate limiting; revoked key activation.
Affiliates	Duplicate conversion; refund reversal; policy version preserved.
SDK	No retry on unsafe mutation without idempotency; token refreshed per request; typed errors; abort behavior.